Lab 3 - Simulation and Odometry

Aim. Drive DiffBot in a small simulator, write the **wheel-odometry** node yourself, and compare your estimate with the simulator's ground truth. Then make the wheels imperfect and watch odometry drift.

Time. 90 min. **Needs.** Only this folder (no compiler).

This folder is a complete project

Everything this lab needs is in this folder. Nothing from the other labs is required.

Requirements: Windows 10/11, PowerShell, pixi (no administrator rights needed; install steps are in README-SETUP.md), and nothing else.

In this folder	
`ws\src\`	the ROS 2 packages; `ws\install\` appears after `pixi run build`
`scripts\`	helper scripts and the automated tests
`code\`	reference solutions / helper programs for this lab
`pixi.toml`	the environment and every `pixi run ...` command; `README.md` this manual; `tools\replay.ps1` re-runs the manual's commands as a self-test

Set it up once. `pixi install` puts ROS 2 Jazzy into `C:\rlabs` (a few GB; pixi's package cache makes the second lab on the same machine fast). Open PowerShell in this folder and run:

```
pixi install
pixi run verify
pixi run build
```

(`pixi run build` compiles this lab's packages into `ws\install`; run it again after every edit in `ws\src`.)

Run one lab at a time, and everything from this folder.

Theory

`diffbot_sim` is a kinematic simulator (no inertia, no friction) in a 10 m x 8 m room. It reads `/cmd_vel`, moves the robot, and publishes what a real robot would: `/joint_states` (wheel angles), `/scan` (360-ray lidar), `/map` and, for checking only, `/ground_truth`. It stops the robot 0.5 s after the last command (watchdog), so a command must be repeated.

With wheel radius `r = 0.05 m` and wheel separation `L = 0.30 m`:

Direction	Formula
command -> wheel speeds	`w_left = (v - w L/2) / r`, `w_right = (v + w L/2) / r`
wheel angle change -> distance	`d_left = r * delta_phi_left`, `d_right = r * delta_phi_right`
distance -> robot motion	`ds = (d_left + d_right) / 2`, `dtheta = (d_right - d_left) / L`
position update (midpoint heading)	`x += ds cos(theta + dtheta/2)`, `y += ds sin(theta + dtheta/2)`, `theta += dtheta`

Frames: `map --(static, = start pose)--> odom --(odometry node)--> base_footprint -> base_link -> wheels, lidar_link`. `/odom` starts at (0, 0) in frame `odom`; `/ground_truth` is in frame `map`, where the robot starts at **(1, 1)**. So *truth = odom + (1, 1)* when odometry is perfect. Heading from a quaternion (z, w): `yaw = 2 * atan2(z, w)`.

Procedure

1. T1 - start the simulator (RViz2 opens: robot, laser points, room walls).

```
pixi run demo-sim
```

2. T2 - what the simulator publishes.

```
pixi run ros node list --no-daemon --spin-time 5
pixi run ros topic list --no-daemon --spin-time 5
```

3. T3 - your own odometry (the exercise). Open `ws\src\diffbot_odometry\exercises\odometry_template.py` and complete the four `TODO`s using the formulas above (the solution is at the end of this page). Run it; it publishes `/my_odom`. With the TODOs still empty it runs without errors and publishes nothing. Here the finished reference file is used:

```
pixi run python code\odometry_solution.py
```

4. T2 - drive. The three commands are: forward 0.8 m, turn left about 1.5 rad, forward 0.8 m (arguments: linear m/s, angular rad/s, seconds).

```
pixi run drive 0.2 0.0 4
pixi run drive 0.0 0.5 3
pixi run drive 0.2 0.0 4
```

5. T2 - compare the three poses. Compute `yaw = 2 atan2(z, w)` from each orientation.

```
pixi run ros topic echo /odom --no-daemon --once --spin-time 4 --field pose.pose
pixi run ros topic echo /my_odom --no-daemon --once --spin-time 4 --field pose.pose
pixi run ros topic echo /ground_truth --no-daemon --once --spin-time 4 --field pose.pose
```

Expected: `/odom` and `/my_odom` identical; `/ground_truth` = the same plus (1, 1).

6. Break the odometry on purpose. Stop T1 and T3 with Ctrl+C.

```
# Ctrl+C in T1 and T3
```

Restart T1 with a wheel that is really 3 % off. The robot now moves with the imperfect wheel, but the encoders (`/joint_states`) still report the ideal rotation - exactly like a real robot. Repeat steps 4 and 5 (without T3).

```
pixi run ros launch diffbot_bringup sim.launch.py rviz:=true wheel_bias:=0.03
```

```
pixi run drive 0.2 0.0 4
pixi run drive 0.0 0.5 3
pixi run drive 0.2 0.0 4
```

```
pixi run ros topic echo /odom --no-daemon --once --spin-time 4 --field pose.pose
pixi run ros topic echo /ground_truth --no-daemon --once --spin-time 4 --field pose.pose
```

Error = distance between `/odom` (x, y) and `/ground_truth` minus (1, 1), and the difference of yaw. Try `wheel_noise:=0.05` (random slip) instead of `wheel_bias:=0.03` too.

Observations (recorded runs of the same three drives; yours differ by a few mm)

Setting	odometry says (x, y, yaw)	truth - (1, 1)	error
perfect wheels	0.844, 0.799, 1.516	0.844, 0.799, 1.516	0.000 m, 0.000 rad
`wheel_noise:=0.05` (random slip)	0.844, 0.799, 1.515	0.857, 0.788, 1.483	0.017 m, 0.032 rad
`wheel_bias:=0.03` (right wheel 3 % off)	0.837, 0.777, 1.523	0.739, 0.821, 1.704	0.107 m, 0.181 rad

Random errors partly cancel; a constant error does not. That is why real robots fuse odometry with other sensors.

Verification

```
pixi run test-class3
```

15 checks: `/joint_states`, `/scan` (360 rays), `/odom`, `/ground_truth` publish; the TF chain `map -> odom -> base_footprint -> base_link -> lidar_link` resolves; after 0.8 m forward and a 1.5 rad turn the odometry is within 3 cm and 0.03 rad of the truth; the robot stops by itself when commands stop.

Solution to the exercise (read after you tried)

```
d_left  = (left  - self.prev_left)  * WHEEL_RADIUS       # TODO 1
d_right = (right - self.prev_right) * WHEEL_RADIUS
ds      = (d_left + d_right) / 2.0                        # TODO 2
dtheta  = (d_right - d_left) / WHEEL_SEPARATION
self.x += ds * math.cos(self.theta + dtheta / 2.0)        # TODO 3
self.y += ds * math.sin(self.theta + dtheta / 2.0)
self.theta += dtheta
# TODO 4: Odometry(); header.frame_id "odom"; child_frame_id "base_footprint"; position x, y;
#         orientation z = sin(theta/2), w = cos(theta/2); self.pub.publish(odom)
```

The complete tested file is `code\odometry_solution.py`.

Questions

1. Why does the simulator need the 0.5 s watchdog and why does `drive` publish at 20 Hz?
2. In the table, which error is larger for the same 3 % imperfection: position or heading? Why does a heading error make later position errors worse?

3. Replace the midpoint heading by plain Euler (`x += ds*cos(theta)`). How much worse is the result after driving a full circle?
4. Why is `map -> odom` a *static* transform here, and what would normally provide it on a real robot?

If it does not work

Symptom	Fix
`drive` prints "nothing is listening on /cmd_vel"	T1 is not running yet (wait 20-30 s)
Robot moves less than expected	the drive started before ROS discovery finished; just repeat
`/my_odom` stays empty	TODOs not complete, or T1 not running; `ros topic list` should show `/joint_states`